Desenmarañando xv6

Posible organización del trabajo

Hoy:

1. Escribir un programa de usuario en `pingpong.c` que imprima sólo *ping* por consola
2. Crear la llamada a sistema `sem_init` que no tome ningún parámetro y que devuelva 0
3. Extender el programa `pingpong` para que llame a `sem_init` e imprima el resultado
4. Extender `sem_init` y pingpong para que reciba un argumento

Luego:

1. Implementar todas las syscalls con un solo semáforo, que accedan a su valor usando spinlocks
2. Extender las syscalls para representar múltiples semáforos
3. Completar las syscalls con la lógica de dormir procesos hasta que el semáforo esté disponible

Primer programa de usuario

uptime.c

uptime es una llamada al sistema que devuelve la cantidad de ticks desde el inicio del sistema.

Tomamos como ejemplo wc.c y construimos nuestro primer programa

* ¿Dónde lo guardamos?

* ¿Qué pasos extra hay que seguir para poder ejecutarlo?

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
 printf("uptime %d\n", uptime());
 exit(0);
}
```

Desenredando uptime

La función “uptime” como tal sólo está declarada en user.h, pero no está implementada

SEARCH

uptime

10 results in 6 files - Open in editor

syscall.c kernel (3)

```
extern uint64 sys_uptime(void);
[SYS_uptime]  sys_uptime,
[SYS_uptime]  sys_uptime,
```

syscall.h kernel (1)

```
#define SYS_uptime 14
```

sysproc.c kernel (1)

```
sys_uptime(void)
```

user.h user (1)

```
int uptime(void);
```

usys.pl user (1)

```
entry("uptime");
```

usys.S user (3)

```
.global uptime
uptime:
li a7, SYS_uptime
```

No podemos llamar a una syscall como una función normal

Motivación de toda esta complejidad

* **Control del flujo de ejecución**: para entrar en espacio kernel, el hardware salta siempre a la dirección configurada en `stvec`, no a cualquier punto arbitrario del código.

* **Validación de argumentos**: el kernel necesita recibir los argumentos de forma controlada y validarlos antes de usarlos.

* **Control de acceso a memoria**: cuando el kernel recibe punteros desde el usuario, valida que las direcciones estén en el rango de memoria de usuario y que el proceso tenga permisos para leer o escribir en esas direcciones.

syscall.h y syscall.c

* Las syscalls se identifican por un número

kernel > C syscall.h > ...

Robert Morris, 4 weeks ago | 3 authors (Frans Kaash

```c
// System call numbers      rsc, 19
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
```

syscall.h y syscall.c

* Las syscalls se identifican por un número

* El arreglo `syscalls` mapea cada identificador a la función que implementa la lógica de la syscall.

kernel > syscall.c > syscall(void)

```c
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_pause]   sys_pause,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
};
```

usys.pl y usys.S

* Cuando se ejecuta el sistema con `make qemu`, se genera el archivo usys.S

* Se genera código assembler RISC-V que implementa un stub de cada syscall en espacio de usuario

* Se guarda el id de la llamada a sistema en el registro a7

* La instrucción `ecall` provoca un **trap** (interrupción) que transfiere el control al kernel.

* El cambio a modo kernel siempre comienza con `usertrap()` donde se maneja la interrupción, y de ahí a `syscall()`

user > ASM usys.S

```
89   getpid:
91     ecall
92     ret
93   .global sys_sbrk
94   sys_sbrk:
95     li a7, SYS_sbrk
96     ecall
97     ret
98   .global pause
99   pause:
100    li a7, SYS_pause
101    ecall
102    ret
103  .global uptime
104  uptime:
105    li a7, SYS_uptime
106    ecall
107    ret
108
```

syscall.c

* La función `syscall` toma el id del registro `a7`

* Llama al manejador correspondiente a esa syscall

* Devuelve el resultado en el registro `a0`

syscall.c > syscall(void)

```c
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if(num > 0 && num < NELEM(syscalls)
    && syscalls[num]) {
    // Use num to lookup the system call function
    // for num, call it,
    // and store its return value in p->trapframe->a0
    p->trapframe->a0 = syscalls[num]();
  } else {
    printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }      rtm, 19 years ago • primitive fork and exit s
}
```

Parámetros

* En RISC-V, al llamar a una función los parámetros se guardan en los registros `a0` hasta `a5` siempre que sea posible.

* Esta parte es igual en una llamada a sistema

* El manejador de la llamada a sistema tiene que leer los parámetros directamente desde los registros

Les queda como ejercicio investigar cómo funciona esta parte

¿Por qué uptime es una syscall?

* La variable global ticks cuenta la cantidad de ticks que ocurrieron desde el inicio del sistema

* Es un recurso compartido que puede ser accedido desde múltiples puntos del código, y por varios CPUs al mismo tiempo

* `uptime` tiene que acceder al valor del recurso de manera segura

kernel > C sysproc.c > sys_uptime(void)

```c
// return how many clock tick interrupts have
// since start.
Windsurf: Refactor | Explain | ×
uint64
sys_uptime(void)        Robert Morris, 15 years
{
  uint xticks;

  acquire(&tickslock);
  xticks = ticks;
  release(&tickslock);
  return xticks;
}
```

Otro ejemplo:

* El scheduler o planificador decide qué proceso ejecutar a continuación

* En `proc.c:438` se itera sobre todos los procesos. Si el proceso está listo para correr (`RUNNABLE`), se ejecuta (`swtch`).

* ¿Qué sucede si otro cpu selecciona el mismo proceso y cambia su estado entre la línea 440 y 446?

kernel > C proc.c > ...

```
422  scheduler(void)
428    for(;;){
437      int found = 0;
438      for(p = proc; p < &proc[NPROC]; p++) {
439        acquire(&p->lock);
440        if(p->state == RUNNABLE) {
441          // Switch to chosen process.  It is the pro
442          // to release its lock and then reacquire i
443          // before jumping back to us.
444          p->state = RUNNING;
445          c->proc = p;
446          swtch(&c->context, &p->context);
447
448          // Process is done running for now.
449          // It should have changed its p->state befo
450          c->proc = 0;
451          found = 1;
452        }
453        release(&p->lock);
454      }
```

* La tabla de procesos `&proc` es compartida por todos los CPUs

* Al **leer y/o escribir** la entrada del proceso en la tabla (`p`) debemos asegurar que ningún proceso la está modificando

* Cada proceso tiene un spinlock guardado en `p->lock` para gestionar el acceso

* `acquire` y `release` protegen una zona crítica de código

kernel > C proc.c > ...

```c
scheduler(void)
  for(;;){
    int found = 0;
    for(p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if(p->state == RUNNABLE) {
        // Switch to chosen process.  It is the pro
        // to release its lock and then reacquire i
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);

        // Process is done running for now.
        // It should have changed its p->state befo
        c->proc = 0;
        found = 1;
      }
      release(&p->lock);
    }
```

Continuará…